Computer Graphics — Problem Set 2 풀이

> 표기: 동차좌표는 열벡터, 변환은 행렬을 **왼쪽에서** 곱한다 (P' = M·P).

---

1. 삼각형 P1 = (1, 0, 1), P2 = (4, 0, 1), P3 = (1, 4, 1)

a) 방향 벡터

- A = P2 − P1 = (4−1, 0−0, 1−1) = **(3, 0, 0)**
- B = P3 − P1 = (1−1, 4−0, 1−1) = **(0, 4, 0)**

b) 외적과 단위 법선

$$A \times B = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ A_x & A_y & A_z \\ B_x & B_y & B_z \end{vmatrix} = (A_yB_z - A_zB_y,\ A_zB_x - A_xB_z,\ A_xB_y - A_yB_x)$$

$$A \times B = (0 \cdot 0 - 0 \cdot 4,\ 0 \cdot 0 - 3 \cdot 0,\ 3 \cdot 4 - 0 \cdot 0) = (0, 0, 12)$$

|A × B| = 12 →

$$N = \frac{A \times B}{|A \times B|} = (\mathbf{0}, \mathbf{0}, \mathbf{1})$$

c) 정점 순서와 전면 방향

- 오른손의 네 손가락을 P1 → P2 → P3 순서(A에서 B 쪽으로 감는 방향)로 감으면 엄지가 가리키는 방향이 A × B, 즉 법선 N의 방향이다.
- 삼각형은 평면 z = 1 위에 있고, +z 쪽(위)에서 내려다보면 P1(1,0) → P2(4,0) → P3(1,4)는 **반시계(CCW)** 방향으로 돈다. 따라서 N = (0, 0, 1)은 +z를 향한다.
- OpenGL은 기본적으로 (glFrontFace(GL_CCW)) 화면에서 반시계로 보이는 면을 **전면(front face)** 으로 정한다. 즉 +z 쪽에서 보는 관찰자에게 이 삼각형은 전면이고, 법선 N이 관찰자 쪽을 향한다.
- 정점 순서를 P1 → P3 → P2로 바꾸면 B × A = −(A × B) = (0, 0, −12)가 되어 법선과 전면 방향이 반대(−z)가 된다. 즉 **정점 배열 순서가 법선 방향과 앞/뒷면을 결정**한다.

---

2. Eye = (0, 0, 4), Center = (0, 0, 0), Up = (0, 1, 0)

a)

- los = Center − Eye = (0, 0, 0) − (0, 0, 4) = **(0, 0, −4)** (카메라가 −z 방향을 봄)
- Eye − Center = (0, 0, 4), |Eye − Center| = 4

$$z_{cam} = \frac{(0, 0, 4)}{4} = (\mathbf{0}, \mathbf{0}, \mathbf{1})$$

(z_cam은 시선의 **반대** 방향, 카메라는 −z_cam을 바라본다.)

b)

$$Up \times z_{cam} = (0,1,0) \times (0,0,1) = (1\cdot 1 - 0\cdot 0,\ 0\cdot 0 - 0\cdot 1,\ 0\cdot 0 - 1\cdot 0) = (1,0,0)$$

크기 1 → **x_cam = (1, 0, 0)**

$$y_{cam} = z_{cam} \times x_{cam} = (0,0,1) \times (1,0,0) = (0\cdot 0 - 1\cdot 0,\ 1\cdot 1 - 0\cdot 0,\ 0\cdot 0 - 0\cdot 1) = (\mathbf{0},\mathbf{1},\mathbf{0})$$

카메라 좌표축이 세계 좌표축과 일치한다 (카메라는 z축 위에서 원점을 보고 있으므로).

c) Eye를 원점으로 옮기는 이동 행렬

$$T_{cam} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -4 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

확인: T_cam · (0, 0, 4, 1)ᵀ = (0, 0, 0, 1)ᵀ. (R_cam = I 이므로 뷰 행렬 V = R_cam · T_cam = T_cam = glTranslatef(0, 0, −4)와 같다.)

---

3. 백페이스 컬링

a) 시선 벡터

V = Eye − P0 = (2, 1, 4) − (2, 1, −1) = **(0, 0, 5)**

b) 내적

N · V = 0·0 + 0·0 + 1·5 = **5** → **양수(+)**

c) 판정

- **N · V > 0**: 법선과 (면 → 카메라) 방향 사이 각이 90° 미만 → 면이 카메라를 향함 → **전면(front-face)**, 렌더링한다.
- **N · V < 0**: 각이 90° 초과 → 면이 카메라를 등짐 → **후면(back-face)**, 컬링(제거)한다.
- (N · V = 0이면 면이 시선과 나란하여 선으로만 보임 — 보통 제거)

N · V = 5 > 0 이므로 이 다각형 면은 **전면이며 렌더링에 포함**된다.

---

4. glOrtho(−4, 4, −2, 2, 1, 9)

a) N = R · S · T 일반식

가시 부피 중심 ((l+r)/2, (b+t)/2, −(n+f)/2)을 원점으로:

$$T = \begin{bmatrix} 1 & 0 & 0 & -\frac{r+l}{2} \\ 0 & 1 & 0 & -\frac{t+b}{2} \\ 0 & 0 & 1 & \frac{f+n}{2} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

크기 (r−l, t−b, f−n)을 2로:

$$S = \begin{bmatrix} \frac{2}{r-l} & 0 & 0 & 0 \\ 0 & \frac{2}{t-b} & 0 & 0 \\ 0 & 0 & \frac{2}{f-n} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

오른손 → 왼손 좌표계 (z 반전):

$$R = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$N = R \cdot S \cdot T = \begin{bmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & -\frac{2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

b) l = −4, r = 4, b = −2, t = 2, n = 1, f = 9

- r − l = 8, r + l = 0 → 2/8 = 0.25, 0
- t − b = 4, t + b = 0 → 2/4 = 0.5, 0
- f − n = 8, f + n = 10 → −2/8 = −0.25, −10/8 = −1.25

$$N = \begin{bmatrix} 0.25 & 0 & 0 & 0 \\ 0 & 0.5 & 0 & 0 \\ 0 & 0 & -0.25 & -1.25 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

(검산: near 평면 z = −1 → −0.25·(−1) − 1.25 = −1, far 평면 z = −9 → 2.25 − 1.25 = 1 ✔)

c) P_eye = (2, −1, −5, 1)

$$P_{clip} = N\,P_{eye} = \begin{bmatrix} 0.25 \cdot 2 \\ 0.5 \cdot (-1) \\ -0.25 \cdot (-5) - 1.25 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.5 \\ -0.5 \\ 0 \\ 1 \end{bmatrix}$$

직교 투영이므로 w = 1 → 원근 나눗셈 후에도 같다:

$$P_{ndc} = \left(\tfrac{0.5}{1}, \tfrac{-0.5}{1}, \tfrac{0}{1}\right) = (0.5,\ -0.5,\ 0)$$

$-1 \le 0.5,\ -0.5,\ 0 \le 1$ 이므로 **CVV 내부에 포함**된다. (원래 점도 −4≤2≤4, −2≤−1≤2, 1≤5≤9로 가시 부피 안에 있었다. z = −5는 near와 far의 정중앙이므로 z_ndc = 0.)

---

5. 뷰포트 변환, glViewport(100, 50, 800, 600)

a) 선형 매핑

[−1, 1] → [left, left+width], [−1, 1] → [bottom, bottom+height]:

$$x' = \frac{width}{2}(x_{ndc} + 1) + left = 400(x_{ndc} + 1) + 100$$

$$y' = \frac{height}{2}(y_{ndc} + 1) + bottom = 300(y_{ndc} + 1) + 50$$

(확인: x_ndc = −1 → 100, 1 → 900; y_ndc = −1 → 50, 1 → 650)

b) P_ndc = (0.5, −0.5, 0.0)

- x' = 400 · (0.5 + 1) + 100 = 400 · 1.5 + 100 = **700**
- y' = 300 · (−0.5 + 1) + 50 = 300 · 0.5 + 50 = **200**

P' = (700, 200)

c) z_ndc를 보존하는 이유

- 여러 3D 점이 같은 화면 픽셀 (x', y')로 투영될 수 있다. 이때 어떤 fragment가 카메라에 가장 가까워 실제로 보여야 하는지는 **깊이**로만 판단할 수 있다.
- 따라서 뷰포트 변환은 z_ndc ∈ [−1, 1]도 깊이 범위(기본 [0, 1], glDepthRange)로 매핑하여 fragment와 함께 보관하고, 래스터화 후 **깊이 테스트(Z-buffer)** 에서 z_buffer에 저장된 값과 비교해 가까운 것만 color buffer에 쓴다.
- z를 버리면 그리는 순서대로 덮어쓰게 되어 뒤에 있는 물체가 앞에 있는 물체를 가리는 잘못된 가시성(hidden surface) 결과가 나온다.

---

6. 행렬 스택

a) glPushMatrix / glPopMatrix

- **glPushMatrix()**: 현재 CTM(스택 top)을 **복사하여** 스택에 하나 더 쌓는다. 지금 상태를 저장해 두는 역할.
- **glPopMatrix()**: 스택 top을 버리고, 바로 아래 저장해 둔 행렬을 다시 CTM으로 **복원**한다. Push 이후 적용한 변환을 취소하는 역할.

실행 추적:

단계	스택 (아래 → 위)	CTM (= top)
초기	I	I
1: glTranslatef	T	T
2: glScalef	T·S	T·S
3: glPushMatrix	**T·S, T·S**	**T·S**
4: A 그림	T·S, T·S	T·S → A에 T·S 적용 ✔
5: glPopMatrix	T·S	T·S
6: glTranslatef	T·S·U	T·S·U

3단계 직후: 스택(아래→위) = **[T·S, T·S]**, 실제 CTM = 맨 위의 **T·S**.

b) 물체 B, P = (1, 1, 0)

B에 실제로 적용되는 변환: **P' = T · S · U · P**

- U·P = (1+3, 1, 0) = (4, 1, 0)
- S·(U·P) = (2·4, 1, 0) = (8, 1, 0)
- T·(…) = (8, 1, 0 − 10) = **(8, 1, −10)**

의도한 결과 T · U · P = (4, 1, −10) 과 **다르다** → B에는 원하지 않은 크기 변환 S까지 적용되었으므로 **의도대로 적용되지 않았다**.

c) 수정

glPushMatrix()를 **S 적용 전(1단계와 2단계 사이)** 으로 옮긴다:

```
        glTranslatef(0, 0, -10);   // T        CTM = T
        glPushMatrix();            //          스택 [T, T]
        glScalef(2, 1, 1);         // S        CTM = T · S
        /* 물체 A를 그린다 */       //          A: T · S ✔
        glPopMatrix();             //          CTM = T (복원)
        glTranslatef(3, 0, 0);     // U        CTM = T · U
        /* 물체 B를 그린다 */       //          B: T · U ✔
```

	Push 시점에 저장된 행렬	Pop이 복원하는 CTM	B에 적용	P 결과
원래 코드	T·S	**T·S**	T·S·U	(8, 1, −10) ✘
수정 코드	T	**T**	T·U	(4, 1, −10) ✔

원인: Push는 그 시점의 CTM을 저장하고 Pop은 그 값으로 되돌릴 뿐이다. 원래 코드는 S를 적용한 **뒤에** Push 했으므로 저장된 상태에 이미 S가 포함되어 있었고, Pop 후에도 S가 남아 B까지 확대되었다. A에만 적용할 변환(S)은 반드시 Push **이후**에 호출해야 한다.

---

7. Z-buffer

a)

- 처리 직후 z_buffer에는 그 픽셀에 지금까지 들어온 fragment들 중 **가장 작은 깊이(카메라에 가장 가까운 표면의 깊이)**, 즉 현재 color_buffer에 기록된 색을 가진 fragment의 깊이가 저장되어 있어야 한다.
- 초기값을 1.0(최대 깊이, far 평면)으로 두는 이유: 처음 들어오는 fragment가 반드시 비교(z_new < z_buffer)를 통과하도록 하기 위해서이다. "아직 아무것도 그려지지 않음 = 무한히 먼 배경"을 의미하며, 이 값이 너무 작으면 실제 물체 fragment가 배경 깊이에 막혀 그려지지 않는다.

b) 주어진 코드 (z_buffer를 갱신하지 않는 오류)

처리 후	비교	color_buffer	z_buffer
초기	-	BLACK	1.0
F1 (RED, 0.65)	0.65 < 1.0 ✔	RED	1.0
F2 (BLUE, 0.25)	0.25 < 1.0 ✔	BLUE	1.0
F3 (GREEN, 0.25)	0.25 < 1.0 ✔	GREEN	1.0
F4 (YELLOW, 0.80)	0.80 < 1.0 ✔	**YELLOW**	**1.0**

z_buffer가 갱신되지 않아 모든 fragment가 통과하고, 가장 마지막(가장 먼) YELLOW가 남는다 → **오류**.

수정 코드:

```
        if (z_new < z_buffer) {
            color_buffer = color_new;
            z_buffer = z_new;        // 추가: 가장 가까운 깊이를 기록
        }
```

처리 후	비교	color_buffer	z_buffer
초기	-	BLACK	1.0
F1 (RED, 0.65)	0.65 < 1.0 ✔	RED	0.65
F2 (BLUE, 0.25)	0.25 < 0.65 ✔	BLUE	0.25
F3 (GREEN, 0.25)	0.25 < 0.25 ✘	BLUE	0.25
F4 (YELLOW, 0.80)	0.80 < 0.25 ✘	**BLUE**	**0.25**

c) F2와 F3 순서 교환 (F1 → F3 → F2 → F4)

처리 후	비교	color_buffer	z_buffer
F1 (RED, 0.65)	0.65 < 1.0 ✔	RED	0.65
F3 (GREEN, 0.25)	0.25 < 0.65 ✔	GREEN	0.25
F2 (BLUE, 0.25)	0.25 < 0.25 ✘	GREEN	0.25
F4 (YELLOW, 0.80)	0.80 < 0.25 ✘	**GREEN**	**0.25**

- 최종 **깊이는 0.25로 같다** (최소 깊이는 순서와 무관).
- 최종 **색은 BLUE → GREEN으로 달라진다.**
- 이유: 비교가 엄격한 부등호 `<`(GL_LESS)이므로 깊이가 **같은** fragment는 통과하지 못한다. 따라서 동일 깊이 0.25에서는 **먼저 처리된 fragment가 이기고** 뒤에 온 것은 버려진다. (`<=`를 쓰면 반대로 나중 것이 이긴다.) 깊이가 같으면 결과가 처리 순서에 의존하며, 이것이 z-fighting의 원인이다.

---

8. glm 벡터 연산, a = (2, 1, −2), b = (1, 2, 2)

a) 외적

$$a \times b = (a_y b_z - a_z b_y,\ a_z b_x - a_x b_z,\ a_x b_y - a_y b_x)$$

$$n_1 = a \times b = (1 \cdot 2 - (-2) \cdot 2,\ (-2) \cdot 1 - 2 \cdot 2,\ 2 \cdot 2 - 1 \cdot 1) = (\mathbf{6}, -\mathbf{6}, \mathbf{3})$$

$$n_2 = b \times a = (2 \cdot (-2) - 2 \cdot 1,\ 2 \cdot 2 - 1 \cdot (-2),\ 1 \cdot 1 - 2 \cdot 2) = (-\mathbf{6}, \mathbf{6}, -\mathbf{3})$$

- 크기: |n1| = |n2| = √(36 + 36 + 9) = √81 = **9**
- 방향: **n2 = −n1** (크기 같고 방향 반대, 반교환성 b × a = −a × b)
- 오른손 법칙: 손가락을 a에서 b로 감으면 엄지가 n1 방향, b에서 a로 감으면 손을 뒤집어야 하므로 엄지가 반대(n2) 방향을 가리킨다.

b) 내적과 정규화

- d1 = n1 · a = 6·2 + (−6)·1 + 3·(−2) = 12 − 6 − 6 = **0**
- d2 = n1 · b = 6·1 + (−6)·2 + 3·2 = 6 − 12 + 6 = **0**

→ 외적 결과는 두 피연산자 **a, b 모두에 수직**이다 (a, b가 이루는 평면의 법선). 그래서 다각형 법선 계산에 외적을 쓴다.

$$u = \frac{n_1}{|n_1|} = \frac{(6, -6, 3)}{9} = \left(\tfrac{2}{3}, -\tfrac{2}{3}, \tfrac{1}{3}\right)$$

$$|u| = \sqrt{\tfrac{4}{9} + \tfrac{4}{9} + \tfrac{1}{9}} = \sqrt{\tfrac{9}{9}} = 1 \ \checkmark$$

c) 표준 단위 벡터

- j × k = (1·1 − 0·0, 0·0 − 0·1, 0·0 − 1·0) = (1, 0, 0) = **i**
- k × i = (0·0 − 1·0, 1·1 − 0·0, 0·0 − 0·1) = (0, 1, 0) = **j**

결합 순서 비교:

- (i × j) × j = k × j = **−i = (−1, 0, 0)**
- i × (j × j) = i × **0** = **(0, 0, 0)** (같은 벡터의 외적은 0)

→ (i × j) × j ≠ i × (j × j): 외적은 **결합법칙이 성립하지 않고**, a)에서 보듯 **교환법칙도 성립하지 않는다** (a × b = −b × a).

법선 계산과의 관계: 다각형 법선은 N = (P2 − P1) × (P3 − P1)처럼 정점 순서로 만든 변 벡터의 외적이므로, 정점 순서(CCW ↔ CW)를 바꾸면 피연산자 순서가 바뀌어 N의 부호가 뒤집힌다. 이는 전면/후면 판정과 백페이스 컬링, 조명 계산 결과를 정반대로 만든다. 따라서 모델링 시 정점 순서를 일관되게(OpenGL 기본은 CCW = 전면) 지정해야 하며, 여러 번 외적을 할 때도 연산 순서를 정확히 지켜야 한다.

---

9. eye = (4, 2, 3), center = (0, 2, 0), up = (0, 1, 0)

a) 카메라 축

- los = center − eye = (0−4, 2−2, 0−3) = **(−4, 0, −3)**
- eye − center = (4, 0, 3), |eye − center| = √(16 + 0 + 9) = 5

$$z_{cam} = \frac{(4,0,3)}{5} = (\mathbf{0.8},\ \mathbf{0},\ \mathbf{0.6})$$

$up \times z_{cam} = (0,1,0) \times (0.8,0,0.6) = (1\cdot 0.6 - 0\cdot 0,\ 0\cdot 0.8 - 0\cdot 0.6,\ 0\cdot 0 - 1\cdot 0.8) = (0.6, 0, -0.8)$

크기 √(0.36 + 0.64) = 1 → **x_cam = (0.6, 0, −0.8)**

$$y_{cam} = z_{cam} \times x_{cam} = (0\cdot(-0.8) - 0.6\cdot 0,\ 0.6\cdot 0.6 - 0.8\cdot(-0.8),\ 0.8\cdot 0 - 0\cdot 0.6) = (\mathbf{0}, \mathbf{1}, \mathbf{0})$$

직교성·크기 확인:

- x·y = 0, y·z = 0, z·x = 0.48 + 0 − 0.48 = 0 → 서로 **직교**
- |x| = |y| = |z| = 1 → **단위 벡터** (정규직교 기저)

b) T_cam, R_cam, V

$$T_{cam} = \begin{bmatrix} 1 & 0 & 0 & -4 \\ 0 & 1 & 0 & -2 \\ 0 & 0 & 1 & -3 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \qquad R_{cam} = \begin{bmatrix} 0.6 & 0 & -0.8 & 0 \\ 0 & 1 & 0 & 0 \\ 0.8 & 0 & 0.6 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

V = R_cam · T_cam: 왼쪽 3×3은 R_cam, 마지막 열은 −R_cam·eye

- x_cam · eye = 2.4 + 0 − 2.4 = 0 → 0
- y_cam · eye = 2 → −2
- z_cam · eye = 3.2 + 0 + 1.8 = 5 → −5

$$V = \begin{bmatrix} 0.6 & 0 & -0.8 & 0 \\ 0 & 1 & 0 & -2 \\ 0.8 & 0 & 0.6 & -5 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

적용 결과:

- V · eye = (0.6·4 − 0.8·3, 2 − 2, 0.8·4 + 0.6·3 − 5) = **(0, 0, 0)** → 카메라 위치가 카메라 좌표계의 **원점**이 된다.
- V · center = (0 − 0, 2 − 2, 0 + 0 − 5) = **(0, 0, −5)** → 바라보는 점이 카메라 좌표계의 **−z축 위, 거리 5**(= |eye − center|)에 놓인다. 즉 카메라는 −z_cam 방향을 바라본다.

c) P = (−1, 4, −2), Q = (8, 2, 6)

P:

- x = 0.6·(−1) − 0.8·(−2) = −0.6 + 1.6 = 1
- y = 4 − 2 = 2
- z = 0.8·(−1) + 0.6·(−2) − 5 = −0.8 − 1.2 − 5 = −7

P_eye = (1, 2, −7) → z < 0 → 카메라 **앞쪽** (시선 방향으로 7만큼 떨어짐)

Q:

- x = 0.6·8 − 0.8·6 = 4.8 − 4.8 = 0
- y = 2 − 2 = 0
- z = 0.8·8 + 0.6·6 − 5 = 6.4 + 3.6 − 5 = 5

Q_eye = (0, 0, 5) → z > 0 → 카메라 **뒤쪽** (보이지 않음)

Q의 기하학적 위치: x_eye = y_eye = 0 이므로 Q는 카메라의 시선축(z_cam 축) 위에 있다. 세계 좌표로 보면 Q − eye = (4, 0, 3) = eye − center 이므로

$$Q = eye + (eye - center) = 2 \cdot eye - center$$

즉 **center, eye, Q는 한 직선 위에 있고, eye는 center와 Q의 중점**이다. Q는 eye를 기준으로 center의 반대편(점대칭 위치)에 같은 거리 5만큼 떨어져 있다 (center는 z_eye = −5, Q는 z_eye = +5).